A Unified Intelligence Layer for Software Supply Chain Governance

Anonymous Author(s)
Submission for double-blind review — SCORED '26

ABSTRACT

Artifacts such as Software Bills of Materials (SBOM), Vulnerability Exploitability eXchange (VEX), and Third-Party Notices (TPN) have advanced transparency but remain insufficient for consistent, scalable, and auditable release decision-making. These artifacts answer narrow, format-specific questions; none provides a decision-ready abstraction for determining release eligibility under combined security, legal, architectural, and AI-risk constraints. We present the Unified Intelligence Layer (UIL), a governance abstraction that transforms heterogeneous supply chain artifacts into structured, machine-interpretable assertions over which policy-as-code can be evaluated. We introduce the concept of *safety relevance* and position UIL as the layer that computes, attaches, and audits safety-relevance claims across five artifact classes. We systematize five complementary artifact classes (SBOM, VEX, TPN, Threat Model Bill of Materials (TMBOM), and AI Vulnerability Exchange (AIVEX)) into a single decision model with an explicit assertion schema, a normalization-and-correlation pipeline, and an auditable evaluation trail. We demonstrate feasibility through a proof-of-concept implementation on real CycloneDX 1.5 and OpenVEX 0.2.0 inputs; characterize scalability on synthetic inputs up to 50,000 components; and report a complementary experiment on seven real production SBOMs (3,099 components). UIL is presented as a governance abstraction rather than a standardized framework: it clarifies what a decision-ready supply chain layer must compute, regardless of which underlying tools produce the input artifacts.

CCS Concepts: Security and privacy -> Software security engineering; Software and application security. Software and its engineering -> Software verification and validation.

Keywords: software supply chain security, SBOM, VEX, TPN, AIVEX, TMBOM, policy-as-code, compliance automation, release governance, risk intelligence, decision governance

1 INTRODUCTION

A modern software release is governed by a half-dozen heterogeneous artifacts (SBOMs, VEX documents, Third-Party Notices, threat models, and AI risk descriptors), each produced by a different tool and owned by a different organizational function. No artifact, and no existing combination of them, yields a unified release decision: a human release manager must still reconcile them manually, introducing inconsistency, audit gaps, and delayed delivery cycles. UIL closes this loop by consuming all five artifact classes, correlating their signals across security, legal, architectural, and AI-risk dimensions, and emitting typed *safety-relevance assertions* (small, machine-evaluable claims like `NON_EXPLOITABLE` or `LICENSE_REVIEW_REQUIRED`) over which a policy engine (e.g., OPA/Rego) can produce a deterministic, auditable release decision. The contribution is the abstraction and the safety-relevance concept; we also include a working prototype, a scalability study to 50,000 components, and an experiment over 3,099 components drawn from seven real production SBOMs.

Software supply chain security has evolved rapidly under the combined pressure of pervasive open-source reuse, third-party dependency growth, and the recent emergence of AI-assisted software generation. Regulatory action, notably U.S. Executive Order 14028 [1] and the European Union Cyber Resilience Act [2], has elevated software transparency from a best-practice concern to a procurement and market-access requirement. Organizations now produce a growing and heterogeneous set of compliance and security artifacts: SBOMs [3], VEX documents [4], Third-Party Notices [5], threat-model documentation, and emerging AI/ML risk descriptors.

Despite this artifact proliferation, software release governance remains largely manual, fragmented, and inconsistent. Common consequences observed in release practice include:

- inconsistent release decisions across products and business units;
- duplicated analysis effort across security, legal, and engineering reviews;
- weak audit traceability between source evidence and final release decisions;
- delayed delivery cycles caused by serial, human-mediated reviews.

This paper makes four contributions. First, we systematize the current software supply chain artifact landscape into five complementary classes and identify five structural gaps that prevent these artifacts from supporting automated release governance (§2–§3). Second, we introduce the concept of UIL, a governance abstraction that computes, attaches, and audits safety-relevance claims through structured assertions suitable for policy-as-code evaluation (§4–§6). Third, we extend the abstraction to AI systems through TMBOM and AIVEX, and compare UIL against adjacent frameworks including SLSA [6], in-toto [7], GUAC [8], and S2C2F [9] (§7–§9). Fourth, we demonstrate feasibility through a proof-of-concept implementation (§8.1–§8.4). We close with limitations, threats to validity, and open research problems (§10–§11).

Scope. UIL is positioned as a governance abstraction, not a new artifact format, a new SBOM dialect, or a competing build-integrity framework. It assumes the existence of SBOM, VEX, and similar artifacts and concerns itself with how to convert their combined signal into a release decision that is consistent, explainable, and auditable. UIL is complementary to provenance-and-integrity frameworks such as SLSA and in-toto: those frameworks attest *how* an artifact was built; UIL evaluates *whether* the resulting artifact may be released.

2 PROBLEM STATEMENT

We frame the governance problem as a gap between artifact visibility and decision intelligence. Current practice exhibits three structural deficiencies and two operational ones.

2.1 Structural Deficiencies

- **Artifact Fragmentation.** Software supply chain artifacts are frequently produced and maintained by different organizational functions and stored in separate systems: engineering owns SBOMs, product security typically owns VEX, and legal or compliance owns TPNs. Where fragmentation persists, reconciliation across artifact streams is largely manual.
- **Absent Semantic Correlation.** Existing schemas focus on specific domains (SPDX, CycloneDX, CSAF, OpenVEX, license metadata) and do not provide a widely adopted mechanism for policy-driven correlation across them. Cross-domain correlation today is performed manually, in spreadsheets, or in vendor-specific knowledge graphs that are not interoperable.
- **No Decision-Ready Abstraction.** Release eligibility is not an inherent property of any single artifact. Current systems cannot evaluate high-level organizational release policies directly against raw, disparate artifacts without deriving contextual claims first.

2.2 Operational Consequences

- **Inconsistent Release Decisions.** In the absence of explicitly applied policy models, release decisions may vary across teams or products for reasons unrelated to genuine risk-profile differences.
- **Brittle Audit Trails.** Many organizations face challenges maintaining a complete and reproducible chain of evidence linking source artifacts to final release decisions.

These five gaps share a root cause: existing artifacts provide *visibility* but not *decision intelligence*. A governance layer operating above them, one that normalizes, correlates, and reasons over their combined signal, is the missing structural component. §3 surveys the five artifact classes; §4 introduces UIL as that layer.

3 SOFTWARE SUPPLY CHAIN ARTIFACT LANDSCAPE

Modern supply chains produce specialized artifacts, each answering a distinct question. We organize them into five classes; Table 1 summarizes their scope and limitations.

Class	Answers	Primary Limitation
SBOM	What components are present?	No risk semantics
VEX	Which vulns are exploitable?	Security-only scope
TPN	What license obligations apply?	Weakly standardized
TMBOM	Where are trust boundaries?	Not widely standardized
AIVEX	What AI/ML risks apply?	Schema actively standardizing in CycloneDX 2.0 [19, 20]

Table 1. The five artifact classes covered by UIL.

3.1 SBOM — Software Bill of Materials

An SBOM is a nested inventory of the components and dependencies that make up a software artifact, typically expressed in SPDX [10] or CycloneDX [11]. SBOMs answer: what components are present? They are the foundation of modern supply-chain transparency, but they do not indicate whether any of those components introduce unacceptable risk under the consuming organization's policy.

3.2 VEX — Vulnerability Exploitability eXchange

VEX documents [4], expressible in CSAF, CycloneDX-VEX, or OpenVEX [12], assert the exploitability status of a known vulnerability in a specific product context. VEX makes vulnerability response tractable by suppressing irrelevant findings, but it does not encode legal, architectural, or AI-specific risk.

3.3 TPN — Third-Party Notices

Third-Party Notices are the legal-disclosure artifacts accompanying distributed software, capturing component-level license terms, attribution requirements, and copyleft obligations. TPN remains weakly standardized relative to SBOM and is rarely correlated with security signals at decision time [13, 14, 15].

3.4 TMBOM — Threat Model Bill of Materials

We adopt the term TMBOM (Threat Model Bill of Materials) to refer to the architectural-context artifact that captures system design assumptions, trust boundaries, data flows, and applied mitigations. TMBOM converts a vulnerability finding from 'a CVE exists in component X' into 'this CVE is unreachable because component X sits behind trust boundary Y.'

3.5 AIVEX — AI Vulnerability Exchange

AIVEX extends the VEX pattern to AI/ML components, describing model behavioral risks, training-data provenance and contamination concerns, inference-time attack surfaces (prompt injection as vehicle per CWE-1427, indirect injection, jailbreaks), and adversarial robustness. The CycloneDX specification community is actively standardizing the schema substrate for AIVEX assertions in CycloneDX 2.0 [19, 20]; we treat AIVEX as a logical role above this emerging standard.

4 THE UNIFIED INTELLIGENCE LAYER

4.1 The Governance Problem UIL Solves

UIL is designed to compute a single property across five artifact classes: whether a given component, vulnerability, license obligation, architectural exposure, or AI behavior is relevant to the safe and policy-compliant release of a software product into a specific deployment context. No single artifact carries this property; it emerges from the correlation of all five in deployment context. UIL is designed to compute, attach, and audit that correlated signal as structured, machine-evaluable assertions. Formally, for a component *c* in deployment context *C*, UIL computes:

$$\mathbf{SR(c, C) = Phi\_P (sig\_S(c,C), sig\_L(c,C), sig\_A(c,C), sig\_AI(c,C))}$$

where sig_S, sig_L, sig_A, sig_AI are signal-extraction functions over security (SBOM+VEX), licensing (TPN), architectural exposure (TMBOM), and AI behavior (AIVEX) artifacts respectively, each emitting typed claims; and

Phi_P is the aggregation function induced by the organization's release policy. The SR function carries no hidden state: given the same artifacts and policy version, it yields the same claim, evidence chain, and confidence. Adding a documented mitigation to TMBOM or AIVEX may only reduce or preserve claim severity, never increase it. Two organizations with different release policies see the same sig sub-claims and diverge only at Phi_P, which is the property that makes UIL portable across governance regimes.

4.2 The Layer

We define UIL as a governance abstraction that operates above the artifact layer and performs four functions:

- **Normalization.** Heterogeneous artifact formats (SPDX vs. CycloneDX, CSAF vs. OpenVEX, prose TPN vs. structured legal data) are mapped into a common internal representation.
- **Identity resolution.** Components are resolved to canonical identities across artifacts, addressing the well-known PURL/CPE/SWID-tag mismatch and version-range ambiguity that breaks naïve correlation.
- **Correlation.** Security, legal, architectural, and AI-risk signals about the same component are joined, weighted, and combined into safety-relevance context.
- **Assertion.** The correlated state is emitted as structured, machine-evaluable safety-relevance claims over which policy-as-code can be evaluated.

4.3 Pipeline Architecture

Heterogeneous artifacts enter and pass through five stages: (1) Normalization maps SPDX, CycloneDX, CSAF, OpenVEX, and prose TPN inputs into a common internal representation; (2) Identity Resolution unifies PURL, CPE, SWID, and content-digest identifiers into canonical component identities; (3) the Correlation Engine extracts typed sub-claims via the signal functions sig_S, sig_L, sig_A, and sig_AI; (4) the Assertion Store records the resulting safety-relevance tuples with evidence pointers and confidence; and (5) the Policy Engine evaluates Phi_P over those assertions to produce release decisions. The Assertion Store is independently queryable, which gives the audit trail its reproducibility property.

Algorithm 1 specifies the hierarchical identity resolution procedure used in the §8.1 prototype:

```
Input:  reference r (PURL, CPE, SWID, or digest)
        normalized component set C, optional digest d
Output: (component, strategy, q_identity)

1:  if d provided: for each c in C with c.digest defined:
2:      if c.digest == d  ->  return (c, DIGEST, 1.00)
3:  for each c in C:
4:      if c.canonical_id == r  ->  return (c, EXACT_PURL, 0.90)
5:      coords_r := normalize(r)   // canonical (type, name, resolved version)
6:      if coords_r != _|_:
7:          if normalize(c.canonical_id) == coords_r
8:              ->  return (c, NORMALIZED_COORDS, 0.75)
9:  for each c in C:
10:     if c.type == coords_r.type and c.name == coords_r.name
11:         ->  return (c, FUZZY_VERSION, 0.50)
12: return (null, NONE, 0.00)
```

The strategy returned by Algorithm 1 directly determines Q_identity for downstream assertions:

Strategy	Q_identity	When used
DIGEST	1.00	Content-addressed digest match
EXACT_PURL	0.90	Canonical PURL exact match
NORMALIZED_COORDS	0.75	Type + name + resolved version match
FUZZY_VERSION	0.50	Type + name match, version range overlap
NONE	0.00	No match; assertion suppressed

Table 2. Identity confidence scores by resolution strategy.

5 ASSERTION MODEL

5.1 Definition

An assertion is a tuple: *(component, context, claim, evidence, confidence, timestamp)*

- **component**: Canonical identity of the subject (e.g., a PURL or content-addressed digest).
- **context**: The deployment, distribution, or release scope under which the claim is evaluated.
- **claim**: The safety-relevance claim itself, drawn from a small closed vocabulary (NON_EXPLOITABLE, LICENSE_COMPLIANT, ELEVATED_THREAT, AI_BEHAVIOR_RISK, etc.).
- **evidence**: Cryptographic or referential pointers to the source artifacts that justify the claim (a VEX statement, a TPN clause, a TMBOM mitigation node, an AIVEX entry).
- **confidence**: Conf(A) = Q_artifact x Q_identity x Q_correlation. Q_artifact is drawn from a provenance lookup (signed CycloneDX 1.5 = 0.95, unsigned = 0.85, CycloneDX 1.2 = 0.70; SLSA levels [6] provide a principled basis). Q_correlation reflects inferential strength: direct VEX = 0.95, inferred reachability = 0.55. Low-confidence claims route to human review.
- **timestamp**: The evaluation time, which matters because exploitability, license obligations, and AI risks evolve.

5.2 Claim Vocabulary

Table 3 lists the core claim types that UIL emits across the five artifact classes.

Claim	Dimension	Meaning
NON_EXPLOITABLE	Security (sig_S)	Vulnerability present but suppressed by VEX context or TMBOM isolation
REMEDIATION_REQUIRED	Security (sig_S)	Exploitable finding requiring patch or mitigation before release
LICENSE_COMPLIANT	Licensing (sig_L)	Component meets license obligations under the given distribution context
LICENSE_REVIEW_REQUIRED	Licensing (sig_L)	Copyleft or attribution obligation requires legal review or remediation
ELEVATED_THREAT	Architecture (sig_A)	Component sits on a trust boundary adjacent to a sensitive data path in the deployment context, as documented in TMBOM
AI_BEHAVIOR_RISK	AI (sig_AI)	Model exhibits adversarial-input or behavioral risk relevant to deployment use
PROMPT_INJECTION_RISK_BOUNDED	AI (sig_AI)	Prompt-injection susceptibility (CWE-1427) mitigated by TMBOM-defined input controls

Table 3. Core safety-relevance claim vocabulary emitted by UIL.

The PROMPT_INJECTION_RISK_BOUNDED claim carries an inherently higher verification burden than security or licensing claims: input-control adequacy is an organizational assertion. UIL treats this claim as requiring *double evidence pointers* (both an AIVEX entry documenting the susceptibility and a TMBOM mitigation node documenting the control) and the prototype enforces this in the correlation engine by requiring both sig_AI and sig_A evidence before emitting the claim with confidence above 0.5.

5.3 Example Assertions

- *openssl@3.0.7* in context "enclave-isolated TLS terminator" has claim NON_EXPLOITABLE for CVE-2023-XXXX, evidenced by VEX statement V-1023 and TMBOM trust-boundary B-04, with high confidence.
- *lib-foo@2.1* in context "binary distribution to external customers" has claim LICENSE_REVIEW_REQUIRED under copyleft obligations, evidenced by TPN clause T-77, with high confidence.

• *vendor-sdk@4.2* in context "network-facing service" has claim ELEVATED_THREAT under current deployment topology, evidenced by TMBOM exposure node E-12, with medium confidence.

• *llm-component-X* in context "customer-facing chat" has claim PROMPT_INJECTION_RISK_BOUNDED (CWE-1427) under defined input class, evidenced by AIVEX entry A-09 and TMBOM mitigation M-03, with medium confidence.

6 POLICY-DRIVEN RELEASE GOVERNANCE

Once assertions exist, release governance reduces to evaluating a set of policy rules over those assertions. We adopt the policy-as-code model familiar from OPA/Rego [17]. A policy decision over a release R is the result of evaluating every applicable rule over the assertions associated with R. Rules can:

• approve a release automatically when all applicable assertions clear policy thresholds;

• block a release when one or more assertions cross block thresholds;

• require remediation, holding the release pending an updated assertion;

• require formal risk acceptance, routing the release to a designated approver.

Listing 1 shows an excerpt from the strict-veto Rego policy used by the §8.1 prototype:

```
package uil.release
min_confidence := 0.4
block_reasons[reason] {
  some a in input.assertions
  a.claim == "REMEDIATION_REQUIRED"
  reason := sprintf("block: %s", [a.component])
}
risk_acceptance_reasons[r] {
  some a in input.assertions
  a.claim == "ELEVATED_THREAT"
  r := a.component
}
decision := "BLOCK"           if { count(block_reasons) > 0 }
decision := "RISK_ACCEPT_REQ" if { count(block_reasons) == 0
                                   count(risk_acceptance_reasons) > 0 }
decision := "APPROVE"         if { count(block_reasons) == 0
                                   count(risk_acceptance_reasons) == 0 }
```

Listing 1. Excerpt from policy/release.rego (strict-veto Phi_P).

Three properties of this model are worth emphasizing. First, the policy operates over assertions, not over raw artifacts. Second, every decision is reproducible: given the same assertion set and the same policy version, the same decision is reached. Third, policy evaluation runs locally against the Assertion Store at each consumer's site; UIL specifies what each consumer must compute, not a central authority that computes it for them.

7 EXTENDING GOVERNANCE TO AI SYSTEMS

Traditional supply chain governance is insufficient for AI-driven systems. A model artifact behaves quite differently from a library: its risk profile depends on training-data provenance, fine-tuning history, and runtime input distributions in ways that no SBOM or VEX can capture. We extend the artifact landscape through two emerging constructs.

7.1 TMBOM

TMBOM captures system design assumptions, trust boundaries, applied architectural mitigations, and the threat exposure context against which other risk signals must be interpreted. TMBOM is what makes a vulnerability assessment defensible: a CVE in an isolated component is a different release decision from the same CVE in an internet-facing component, and TMBOM is the artifact that distinguishes them.

7.2 AIVEX

AIVEX extends the VEX pattern to AI/ML components. It captures model behavioral risks, training-data provenance concerns, inference-time attack surfaces (prompt injection as vehicle per CWE-1427, indirect injection, jailbreaks), and adversarial robustness measurements. Combined with TMBOM, AIVEX allows UIL to issue assertions over AI behavior in deployment context, for example asserting that a model is acceptable for an internal data-classification task but not for a customer-facing summarization task, on the basis of the same underlying model card but different TMBOM exposure.

The CycloneDX specification community is actively developing the schema substrate that AIVEX requires. CycloneDX issue #956 [19] establishes that AI behavioral risk assertions (prompt-injection exploitability, agent abuse, model behavior drift) map onto the CycloneDX 2.0 vulnerability and risk objects using CWE-1427 as the weakness anchor and MITRE ATLAS alongside ATT&CK; as the threat classification taxonomy. CycloneDX issue #333 [20] is developing a complementary AI-specific evidence taxonomy covering prompt transcripts, behavioral test results, retrieval traces, tool-call traces, and model-artifact integrity checks. The CycloneDX 2.0 RFC is scheduled for August-September 2026.

UIL's sigma_AI extractor is decoupled from CycloneDX serialization format. It operates over UIL's normalized AIVEX representation (component identity, risk type, CWE anchor, MITRE ATLAS reference, mitigation pointer) rather than raw JSON. If the CycloneDX 2.0 schema changes, only Stage 1's normalization adapter requires updating; the correlation logic, assertion vocabulary, and policy engine are unaffected — the same format-independence that allows UIL to consume SPDX and CycloneDX SBOMs interchangeably.

Together, TMBOM and AIVEX extend UIL beyond software composition into system behavior and operational risk intelligence, a scope traditional supply chain frameworks were never designed to cover.

8 WORKED EXAMPLE AND EVALUATION

8.1 Prototype

The prototype source code is publicly available at `https://anonymous.4open.science/r/uil-prototype` under an Apache 2.0 license. An interactive browser-based demonstration is available at `https://anonymous.4open.science/r/uil-prototype/docs/index.html` — it runs the full five-stage pipeline in the browser, allowing reviewers to inspect assertion generation, confidence computation, and policy evaluation interactively without installing any software.

A proof-of-concept implementation of UIL has been developed in Python and Rego demonstrating that the abstraction is executable end-to-end over real CycloneDX 1.5 SBOMs, OpenVEX 0.2.0 statements, and structured TPN, TMBOM, and AIVEX inputs. Stage 5 ships in two parallel forms: a Rego policy file and an equivalent Python evaluator. Running the prototype on a synthetic but format-conformant input set reproduces the worked example's outcome. Pipeline stages emit seven assertions: three security sub-claims (one NON_EXPLOITABLE for openssl cross-referencing VEX with TMBOM boundary B-04; two REMEDIATION_REQUIRED for CVE-affected components), two licensing sub-claims, one ELEVATED_THREAT from TMBOM, and one PROMPT_INJECTION_RISK_BOUNDED (CWE-1427) from AIVEX. The strict-veto Phi_P policy aggregates these into a single decision (BLOCK) with full evidence pointers. The decision is deterministic: rerunning against the same inputs and policy version produces the same output.

Listing 2 reproduces one assertion verbatim from the prototype's Assertion Store:

```
{
  "component":  "pkg:generic/openssl@3.0.7",
  "context":    "enclave-isolated TLS terminator + gateway",
  "claim":      "NON_EXPLOITABLE",
  "dimension":  "security",
  "evidence":   ["VEX:.../release-r1#CVE-2023-XXXX", "TMBOM:B-04"],
  "confidence": { "q_artifact": 0.85, "q_identity": 0.90,
                  "q_correlation": 0.95, "overall": 0.727, "label": "high" },
  "timestamp":  "2026-05-27T01:58:31Z",
```

```
  "detail":     "CVE-2023-XXXX; TLS handshake path not invoked; boundary B-04."
}
```

Listing 2. One assertion from the prototype Assertion Store, verbatim.

The sig function implementations underlying UIL have received independent commercial and community validation: a production vendor implementation of the security and architectural signal extractors has shipped, and the maintainers of CycloneDX, SPDX, OpenSSF, and OpenChain have engaged substantively with the assertion vocabulary. UIL treats those validations as confirmation that the governance pipeline is implementable without changes to any core SBOM specification.

8.2 Scalability

Table 4 reports end-to-end pipeline performance on synthetic inputs from 10 to 50,000 components, run twice at each size to verify rerun determinism.

Components	Assertions	SBOM size	Runtime (s)	Policy (s)	RSS (MB)
10	6	2.9 KB	0.002	<0.001	19.2
100	34	26.4 KB	0.004	<0.001	19.6
1,000	337	265 KB	0.034	<0.001	23.3
10,000	3,328	2.6 MB	0.661	0.001	58.6
50,000	17,007	13.2 MB	11.110	0.002	227.4

Table 4. Prototype scalability on synthetic inputs. All sizes produce deterministic rerun output.

The pipeline scales to 50,000 components in approximately 11 seconds of single-threaded Python on commodity hardware. Peak resident memory remains under 230 MB. Policy evaluation is essentially free relative to the pipeline (2 ms over 17,007 assertions at N=50,000), confirming that cost concentrates in correlation rather than aggregation, which is the property that makes the policy-aggregation separation of §4.1 architecturally sound.

8.3 Real-SBOM Experiment

Seven publicly available CycloneDX SBOMs were retrieved from the official CycloneDX bom-examples corpus at commit a3f1c9e (accessed May 2026): CERN LHC resource-tooling SBOM, Dropwizard 1.3.15, OWASP Juice Shop v11.1.2, Keycloak 10.0.2, Laravel 7.12.0, Proton Bridge v1.6.3, and ProtonMail Webclient v4. Together these span 3,099 real components across Maven, npm, PyPI, Composer, and Go ecosystems. We overlaid a 4% synthetic-VEX coverage and a minimal TMBOM on each SBOM. Table 5 reports per-project results.

Project	Components	Lic. cov.	sig_S	sig_A	Runtime
CERN LHC VDM Editor	43	100%	1	1	<0.001s
Dropwizard 1.3.15	167	79.0%	7	1	<0.001s
OWASP Juice Shop v11.1.2	840	95.6%	37	1	0.001s
Keycloak 10.0.2	903	96.7%	43	1	0.001s
Laravel 7.12.0	62	100%	3	1	<0.001s
Proton Bridge v1.6.3	201	96.0%	4	1	<0.001s
ProtonMail Webclient v4	883	98.4%	37	1	0.001s
Aggregate (7 projects)	**3,099**	**95.1%**	**132**	**7**	**—**

Table 5. Real-SBOM experiment over seven production CycloneDX SBOMs.

The pipeline processed all 7 projects without modification. Identity resolution: 59% via digest (Q_identity=1.00), 41% via exact PURL (Q_identity=0.90). License coverage ranged from 79.0% to 100%, confirming the artifact-quality heterogeneity concern of §11.

8.4 What Artifact-Only Workflows Cannot Produce

Under artifact-only review of the §8 scenario, four functions inspect four artifact streams in parallel. Each produces a correct artifact for its own dimension; none produces a release decision. What UIL produces that no artifact-only tool can emit, because it depends on signal from two artifacts owned by two functions, is the cross-dimensional assertion set: a NON_EXPLOITABLE assertion citing VEX V-1023 and TMBOM B-04 is not a fact any single artifact tool can emit. Likewise for PROMPT_INJECTION_RISK_BOUNDED (CWE-1427) citing AIVEX A-09 and TMBOM M-03. These cross-dimensional assertions, with audit trails linking each to source evidence, are the new artifact UIL produces, and the artifact release governance has been doing without.

9 COMPARISON WITH RELATED FRAMEWORKS

SLSA [6] specifies build-integrity levels and the provenance attestations needed to achieve them. SLSA asks how the artifact was built; UIL asks whether it should be released. SLSA provenance is a valuable input to UIL assertions but is not itself a governance layer.

in-toto [7] provides a framework for cryptographically attesting steps in a software supply chain. UIL does not replicate in-toto attestations; instead, in-toto attestations can serve as evidence elements within UIL assertions.

GUAC [8] aggregates software supply chain metadata into a queryable knowledge graph. A GUAC-style knowledge layer is a natural substrate for a UIL implementation.

S2C2F [9] prescribes practices and maturity levels for consuming open-source dependencies. S2C2F is a practice framework; UIL is a decision abstraction. They operate at different levels and are compatible.

Sigstore [18] provides signing and transparency infrastructure for software artifacts. Sigstore signatures are evidence inputs to UIL, not competing layers.

Commercial SBOM/SCA platforms have begun incorporating VEX-aware risk prioritization and context-dependent exploitability reasoning. These implementations remain single-dimensional (security-only, via VEX-over-SBOM); to the best of our knowledge, no publicly documented implementation has yet exposed a unified cross-domain assertion model spanning security, license compliance, architecture (TMBOM), and AI behavior (AIVEX), evaluable against executable release policies. UIL is positioned at exactly that gap. Adjacent open-source tools such as Anchore Enterprise, Dependency-Track, and OpenSSF Scorecard occupy related niches in artifact aggregation and trust scoring and are likewise complementary rather than competing.

Structural gap (§2)	SLSA	in-toto	GUAC	Comm. SCA	UIL
Artifact fragmentation	No	No	Part	Part	Yes
Semantic correlation	No	No	Part	Part	Yes
Decision-ready abstraction	No	No	No	No	Yes
Policy-driven release	No	No	Part	Part	Yes
Reproducible audit trail	Part	Part	Part	No	Yes
AI/ML risk integration	No	No	No	Part	Yes

Table 6. Capability comparison against structural gaps (§2). Yes = addressed; Part = partial; No = not addressed.

The pattern across these comparisons is consistent: existing frameworks and platforms largely operate at the artifact and attestation level. UIL operates one level above, treating those frameworks outputs as inputs to a decision pipeline.

What adjacent frameworks cannot express. UIL's assertion layer produces cross-dimensional claims that depend simultaneously on artifacts owned by different organizational functions. A NON_EXPLOITABLE assertion citing both VEX and TMBOM cannot be emitted by any single artifact tool; neither can PROMPT_INJECTION_RISK_BOUNDED, which requires both AIVEX and TMBOM evidence. SLSA, in-toto, GUAC, and SCA platforms all operate within a single artifact namespace. UIL's assertion vocabulary formalizes the cross-artifact inferences release managers currently perform in their heads, inconsistently, without an audit trail — the structural gap Table 6 encodes under decision-ready abstraction.

10 AUDITABILITY AND COMPLIANCE

Auditability is the design goal. The full governance pipeline — Artifacts > Assertions > Policy Evaluation > Decision — is fully recorded. An auditor can trace any release decision back to its policy version, assertion set, and source artifacts. Assertion Store outputs can be signed as in-toto [7] attestations binding the assertion set, policy, and decision into a tamper-evident, independently verifiable package, supporting EO 14028, CRA, HIPAA, and PCI compliance reporting.

11 LIMITATIONS AND OPEN PROBLEMS

- **Artifact-Quality Heterogeneity.** UIL's assertions depend entirely on input quality. Real-world data frequently suffers from incomplete SBOMs, prose-based TPNs, and partial VEX coverage, which degrades decision confidence.
- **Identity Resolution.** Cross-artifact component identification remains unsolved. Overlapping and non-coextensive namespaces (PURL, CPE, SWID, content digests) cause naïve matching to yield false positives and false negatives.
- **Inference Errors in Correlation.** Aggregating signals across security, legal, and AI dimensions introduces algorithmic logic that may itself be flawed, requiring explicit confidence models and manual overrides.
- **Unstandardized AI-Risk Representation.** The CycloneDX 2.0 AIVEX schema [19, 20] is under active development; any current implementation choice remains subject to revision until the RFC cycle closes.
- **Sociotechnical Complexity.** Aligning traditionally isolated functions (security, legal, engineering, compliance) around a shared policy engine is a major operational hurdle.
- **Policy Authoring Gap.** Translating informal compliance policies into executable policy-as-code is a skill most organizations have yet to acquire.

11.1 Threats to Validity

- **Construct validity.** TMBOM and AIVEX are emerging rather than standardized; the schemas exercised by the prototype are experimental. The SR aggregation function is defined operationally; alternative formalizations would yield different outputs.
- **External validity.** The scalability experiment uses synthetic inputs; the real-SBOM experiment covers seven open-source projects. New artifact classes (hardware-attestation, AI training-data provenance) may not fit cleanly into the sig structure.
- **Internal validity.** Algorithm 1's heuristics, the per-strategy Q_identity values, and the multiplicative confidence composition are design choices supported by the prototype but not empirically calibrated.

11.2 Threat Model and Trust Assumptions

UIL operates above an input layer of supplier-authored artifacts, so input integrity is a first-order concern. Adversary capabilities include: a malicious supplier issuing self-serving VEX; an intermediary tampering with artifacts in transport; a typosquatted package whose PURL fuzzy-matches a legitimate identity; or a compromised internal tool generating malformed TMBOM or AIVEX. The policy engine and Assertion Store are assumed trusted; crypto primitives are unbroken.

Conflicting assertions and assertor authority. When a supplier VEX asserts NON_EXPLOITABLE but internal TMBOM indicates the component is internet-facing and exposed, UIL emits both assertions. Under strict-veto Phi_P, the ELEVATED_THREAT from TMBOM routes to risk acceptance regardless of the supplier VEX claim, because the TMBOM carries higher Q_artifact (internal analysis > supplier self-attestation). A natural extension is Q_artifact = Q_source x Q_provenance, where Q_source reflects the assertor authority tier and Q_provenance reflects cryptographic attestation strength via SLSA [6] or Sigstore [18]; this is identified as priority future work.

UIL composes with existing provenance frameworks: Sigstore [18] addresses authorship integrity; in-toto [7] attestations bind artifacts to build steps; SLSA [6] provenance levels constrain Q_artifact.

12 CONCLUSION

Software supply chain governance is transitioning from artifact-based visibility toward intelligence-driven decision systems. SBOM, VEX, TPN, and related artifacts provide essential transparency but cannot, individually or in combination, support consistent and scalable release decisions. The Unified Intelligence Layer offers a structured abstraction that transforms fragmented artifacts into correlated, machine-interpretable safety-relevance assertions over which policy-as-code can be evaluated; by extending the same abstraction across TMBOM and AIVEX, UIL covers traditional software and AI-driven systems within one model, and is designed to sit above the CycloneDX 2.0 AI behavioral risk schema currently under standardization [19, 20].

Whether the community standardizes on a single UIL-shaped layer or implements several complementary ones is a secondary question. The primary contribution of this paper is to make explicit what any such layer must compute: a normalized, correlated, policy-evaluable assertion over the combined safety-relevance signal of the five artifact classes. The prototype demonstrates that this computation is feasible, deterministic, and space-efficient at enterprise scale. The open problems identified in §11 (identity resolution accuracy, AI-risk schema stabilization, and real-world artifact corpora evaluation) define the research agenda for turning this governance abstraction into a production-grade standard.

REFERENCES

[1] The White House. 2021. Executive Order 14028: Improving the Nation's Cybersecurity. May 12, 2021.

[2] European Union. 2024. Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act).

[3] NTIA. 2021. The Minimum Elements for a Software Bill of Materials (SBOM). U.S. Department of Commerce.

[4] CISA. 2023. Vulnerability Exploitability eXchange (VEX) — Use Cases. Cybersecurity and Infrastructure Security Agency.

[5] Open Source Initiative. License compliance documentation guidance for distributed software.

[6] OpenSSF. 2024. SLSA: Supply-chain Levels for Software Artifacts, v1.0. https://slsa.dev.

[7] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. 2019. in-toto: Providing farm-to-table guarantees for bits and bytes. USENIX Security Symposium.

[8] OpenSSF. GUAC: Graph for Understanding Artifact Composition. https://guac.sh.

[9] OpenSSF. 2023. Secure Supply Chain Consumption Framework (S2C2F).

[10] Linux Foundation. SPDX Specification, v3.0.

[11] OWASP. CycloneDX Bill of Materials Specification.

[12] OpenSSF. OpenVEX Specification, v0.2.0.

[13] Anonymous. 2026. TPN Intelligence Framework: Automated License and Compliance Risk Analysis from Third-Party Notices. [withheld for review]

[14] Anonymous. 2026. Why Third-Party Notices Are Breaking at Scale and What the Ecosystem Needs Next. [withheld for review]

[15] Revenera. Lessons Learned from Analyzing Large-Scale Third-Party Notices.

[16] Anonymous. 2026. Practical Governance of AI/ML Exploitability and Behavioral Risks via TMBOM and AIVEX. [withheld for review]

[17] Sandall, T. 2018. Open Policy Agent: Policy-based control for cloud-native environments. CNCF / linuxfoundation.org.

[18] Newman, Z., Meyers, J. S., and Torres-Arias, S. 2022. Sigstore: Software signing for everybody. ACM CCS.

[19] CycloneDX Specification. 2026. Issue #956: AI Vulnerability Exploitability eXchange (AIVEX). https://github.com/CycloneDX/specification/issues/956.

[20] CycloneDX Specification. 2026. Issue #333: AI-specific evidence taxonomy for vulnerabilities. https://github.com/CycloneDX/specification/issues/333.